Lab 09: Transaction Management

Use lab12 folder

- Change directory into **lab12** directory
- Save the file and extract it under Lab folder
- Remember to start by creating a sanjose database to use.

A. Transaction Management

A1. Create sanjose Database with mysql

For this exercise lab, create sanjose database using mysql (if needed). Then, create a table, named ACCT using `cr_db.sql` file.

```
mysql> CREATE DATABASE sanjose;
mysql> USE sanjose;
mysql> CREATE TABLE ACCT ( ID  VARCHAR(10), BAL INT);
mysql> INSERT INTO ACCT VALUES('A', 100);
mysql> INSERT INTO ACCT VALUES('B', 1000);
mysql> COMMIT;
```

A2. Transaction with Python programming

First open tx_lab.py file. Then, change the following information:

- localhost: if you are using MySQL on GCP, replace it by IP address. Otherwise, leave as it is.
- USER: database username
- PASS: database password

Run tx_lab.sql file

```
$ python3 tx_lab.sql
```

```
import mysql.connector

try:
    conn = mysql.connector.connect(host='localhost',
                                   database='sanjose',
                                   user='USER',
                                   password='PASS')

    conn.autocommit = False
    cursor = conn.cursor()
    # Deposit to account A
    sql_update_query = """Update acct set bal = bal + 100 where id =
'A'"""
    cursor.execute(sql_update_query)
```

```
        # Withdraw from account B
        sql_update_query = """Update acct set bal = bal - 100  where id =
'B'"""
        # Withdraw from account B with Error
        #sql_update_query = """Update acct set bal = bal - 100  where d =
'B'"""
        cursor.execute(sql_update_query)
        print ("Record Updated successfully ")

        #Commit your changes
        conn.commit()

    except mysql.connector.Error as error :
        print("Failed to update record to database rollback:
    {}".format(error))
        #reverting changes because of exception
        conn.rollback()
    finally:
        #closing database connection.
        if(conn.is_connected()):
             cursor.close()
             conn.close()
             print("connection is closed")
```

You can check ACCT table BAL if the python program run correctly.

Next, remove comment (#) of "withdraw from account B with error" (i.e., `sql_update_query = """Update acct set bal = bal - 100  where d = 'B'"""`) part, while adding comment (#) into "withdraw from account B".

Run tx_lab.py file again. Then, check what happened in ACCT table (BAL).

A3. Simulation of Multiple Transaction with two mysql sessions

In this lab, you are going to simulated the situation of multiple transactions accessing the same data. One transaction (tx1.sql) is updating ACCT.BAL, while the 2nd transaction (tx2.sql) is selecting ACCT.BAL. See what happens.
Open two mysql connections: one is for tx1 using `tx1.sql`, while 2nd is for tx2 using `tx2.sql`.

Sequence	Tx 1 (tx1.sql)	Tx 2 (tx2.sql)
0	`Connect mysql`	`Connect mysql`
1	`mysql> SET AUTOCOMMIT = 0`	`mysql> SET AUTOCOMMIT = 0`
2	`mysql> UPDATE ACCT SET BAL = 100 WHERE ID = 'A';`	
3		`mysql> SELECT * FROM ACCT;` (check BAL if you can see updated bal)
4	`mysql> COMMIT;`	
5		`mysql> SELECT * FROM ACCT;` (check BAL if you can see updated bal)
6		`mysql> COMMIT;`

7		`mysql> SELECT * FROM ACCT;` (check BAL if you can see updated bal)

Discuss why Tx 2 can't or can see the updated value.

B. Consistency with Isolation in MySQL

In this lab, you are going to demonstrate the following isolation levels:

- [B1] Read Uncommitted: iso1_tx1.sql and iso1_tx2.sql
- [B2] Read Committed: iso2_tx1.sql and iso2_tx2.sql
- [B3] Repeatable Read: iso3_tx1.sql and iso3_tx2.sql

B1. Read Uncommitted

In this lab, Tx 1 is updating the data, while Tx 2 which is "Read Uncommitted" isolation level is selecting the data.

Open two mysql connections: one is for tx1 using `iso1_tx1.sql`, while 2nd is for tx2 using `iso1_tx2.sql`.

Sequence	Tx 1 (iso1_tx1.sql)	Tx 2 (iso1_tx2.sql)
0	`Connect mysql`	`Connect mysql`
1		`mysql> SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;`
2	`mysql> SET AUTOCOMMIT = 0`	
3	`mysql> SELECT * FROM ACCT;`	
4	`mysql> UPDATE ACCT SET BAL = 200 WHERE ID = 'A';`	
5	`SELECT * FROM ACCT;`	
6		`mysql> SELECT * FROM ACCT;`
7	`mysql> ROLLBACK;`	
8	`mysql> SELECT * FROM ACCT;`	
9		`mysql> SELECT * FROM ACCT;`

In Step 6, can Tx 2 see the updated BAL? Compare with a result in A3. Why?

B2. Read Committed

In this lab, Tx 1 is updating the data, while Tx 2 which is "Read Committed" isolation level is selecting the data.

Open two mysql connections: one is for tx1 using `iso2_tx1.sql`, while 2nd is for tx2 using `iso2_tx2.sql`.

Sequence	Tx 1 (iso2_tx1.sql)	Tx 2 (iso2_tx2.sql)
0	Connect mysql	Connect mysql
1		mysql> SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
2	mysql> SET AUTOCOMMIT = 0	
3	mysql> SELECT * FROM ACCT;	
4	mysql> UPDATE ACCT SET BAL = 200 WHERE ID = 'A';	
5	mysql> SELECT * FROM ACCT;	
6		mysql> SELECT * FROM ACCT;
7	mysql> COMMIT;	
8	mysql> SELECT * FROM ACCT;	
9		mysql> SELECT * FROM ACCT;

In Step 6, can Tx 2 see the updated BAL? Compare with a result in B1. Why? Also, can Tx 2 see the updated BAL after COMMIT? Why?

B3. Repeatable Read

In this lab, Tx 1 is updating the data, while Tx 2 which is "Repeatable Read" isolation level is reading the data if Tx1 changes affect it or not.

Open two mysql connections: one is for tx1 using `iso3_tx1.sql`, while 2nd is for tx2 using `iso3_tx2.sql`.

Sequence	Tx 1 (iso3_tx1.sql)	Tx 2 (iso3_tx2.sql)
0	Connect mysql	Connect mysql
1	mysql> SET AUTOCOMMIT = 0	mysql> SET AUTOCOMMIT = 0
2		mysql> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
3		mysql> SELECT ID FROM ACCT WHERE BAL = 200;
4	mysql> SELECT * FROM ACCT;	
5	mysql> INSERT INTO ACCT VALUES ('C', 200);	
6	SELECT * FROM ACCT;	
7		mysql> SELECT ID FROM ACCT WHERE BAL = 200;
8	mysql> COMMIT;	
9	mysql> SELECT * FROM ACCT;	
10		mysql> SELECT ID FROM ACCT WHERE BAL = 200;

Check Step 7 and 10 of Tx 2. Before and after COMMIT in Tx 1, can Tx 2 read 'C' or not? Why?

Exercise: Transaction Management

After you complete Exercise Lab 12, answer the following questions:

1. In A2, after running tx_lab.py file again. what happened in ACCT table (BAL)?

2. In A3, why Tx 2 can't or can see the updated value?

4. In B1- Step 6, can Tx 2 see the updated BAL? Compare with a result in A3. Why?

5. In B2 - Step 6, can Tx 2 see the updated BAL? Compare with a result in B1. Why? Also, can Tx 2 see the updated BAL after COMMIT? Why?

6. In B3, check Step 7 and 10 of Tx 2. Before and after COMMIT in Tx 1, can Tx 2 read 'C' or not? Why?

Then, submit your answers on Canvas.